

Introduction

A custom TAGE-style predictor designed to outperform both Gshare and the Tournament predictors. In this report, the intuition, implementation and parameters of custom predictor will be discussed.

Intuition

The custom predictor is inspired by the TAGE family of predictors. This predictor is a good fit for our project because it has been proven in industrial products. The core idea is that the tables represent increasing levels of complexity: more complex tables handle more complex branch behaviours. We customized the hash functions, bit widths, and update mechanisms to optimize performance on the given traces.

- Use one small, simple base predictor (a bimodal table).
- 4 multiple tagged predictor tables that each use different lengths of global history
- For each branch, search the tagged tables from longest to shortest history to find a matching provider entry
- Use u bits to allocate new entries where they matter most and de-allocate unhelpful entries.

This architecture captures correlations with different history lengths. This is particularly powerful for complex workloads with wide range of dependencies between branches.

Data structures

1. Base bimodal predictor
 - 2048 entries of 2-bit saturating counters
 - $\text{Index} \Rightarrow (\text{pc} \wedge (\text{pc} \gg 2)) \& \text{mask}$
2. Tagged TAGE tables
 - T0: 2048 entries, history length 4, tag bits 8, 1 u bit, 4 bits signed saturating counters
 - T1: 1024 entries, history length 12, tag bits 8, 4 bits signed saturating counters
 - T2: 1024 entries, history length 36, tag bits 9, 4 bits signed saturating counters
 - T3: 512 entries, history length 60, tag bits 9, 4 bits signed saturating counters
3. Global history and folded histories
 - custom_ghr: 60-bit history.
 - index and tag hashing functions for different history lengths.
4. Usefulness aging
 - custom_use_age: we periodically reset the u bits once in 262144 branches

Prediction algorithm

Choose the prediction of the longest table hits. If there is no hit use bimodal.

Training algorithm

1. Update base bimodal predictor
2. Identify provider
3. Update provider entry if it exists
4. Update usefulness bit u of the provider if
 - If $\text{pred} == \text{outcome}$ and the counter is strong ($\text{pctr} \geq +2$ or $\text{pctr} \leq -3$), set $u = 1$
 - If $\text{pred} \neq \text{outcome}$ and the counter is strong, set $u = 0$.
5. Decide if allocation is needed:
 - If $\text{pred} \neq \text{outcome} \rightarrow \text{need_alloc}$
6. Allocate new entries in 2 of the deeper tables when needed
7. Periodic reset of u bits once in 262144 branches
8. Update global history

Intuition behind parameter choices

Our custom predictor uses four tagged tables with history lengths of 4, 12, 36, and 60 bits. The 4-bit history captures very short local patterns like simple loops and if-then-else behavior; 12 bits covers medium loops and nested control; 36 bits targets more complex, medium-long correlations; and 60 bits reaches very long-range, irregular but repeatable patterns. Table capacities follow this progression: T0 is the largest (2048 entries) because short histories apply to the widest variety of branches; T1 and T2 have 1024 entries each to balance capacity; and T3 is smaller (512 entries) since long-history matches are less common. Each entry stores a tag of 8-9 bits to reduce aliasing and false matches, with slightly larger tags (9 bits) on the longer-history tables to protect precision. Predictions use 4-bit signed counters in the range $[-4, +3]$, which provide finer confidence than 2-bit counters. Finally, each entry has a usefulness bit u that approximates how often it has been helpful; we periodically reset these u bits to free space for new, potentially more useful entries and to prevent old, stale entries from occupying the tables indefinitely.

Hardware budget calculation

Base bimodal table: $2^{11} = 2048$ 2-bits entries $\rightarrow 2048 \times 2 \rightarrow 4,096$ bits.

T0: 2048 entries 14-bits $\rightarrow 2048 \times 13 = 26,624$ bits

T1: 1024 entries $\rightarrow 1024 \times 13 = 13,312$ bits

T2: 1024 entries $\rightarrow 1024 \times 14 = 14,336$ bits

T3: 512 entries $\rightarrow 512 \times 14 = 7,168$ bits

64k total and other miscellaneous registers are already less than 1024 bits.

Performance evaluation

Table 1. Performance evaluation

	U1 MP rate	U2 MP rate	U3 MP rate	U4 MP rate
Gshare	33.654	101.729	19.608	10.030
Tournament	29.451	94.841	15.657	6.494
Custom	28.213	96.121	11.024	6.243

Conclusion

We able to beat both predictors within the limit of budget with our custom predictor. With another algorithms and parameter optimizations it might perform even better.